

Month 1 — Computer Architecture & Assembly

(24-Day Micro-Plan)

Audience: motivated beginner-intermediate self-taught engineer with prior intro programming.
Cadence: **6 days/week, 1-2h/day**. Target: **x86-64 (Intel syntax) on Linux/WSL/macOS** using **NASM + GCC/Clang + Binutils + GDB**. AArch64 notes are included where relevant.

Exit criteria (Day 24): (1) Read small disassemblies; (2) Write non-trivial assembly interoperating with C; (3) Explain source → ELF → process → CPU → caches; (4) Ship a syscall-only **grep-lite** with an asm hot loop.

Setup (once, 30–60 min)

- **Linux/WSL:** `sudo apt update && sudo apt install build-essential gcc g++ make gdb binutils nasm strace linux-tools-common`
- **macOS:** `brew install llvm nasm binutils` (use `lldb` and `gobjdump/objdump`).
- **Verify:** `nasm -v`, `objdump -v`, `readelf --version`, `gdb --version`, `perf --version` (Linux).

Folders

```
~/month1-arch/  
  week1-repr/  
  week2-asm/  
  week3-cache/  
  week4-os/
```

Keep a lab journal (`journal.md`) with commands, measurements, and retrospectives.

How to use this plan: Each day includes **Learn** (10–35m), **Hands-On** (40–90m), and an optional **Plan-breaker** (10–20m) to fight plan-rigidity (productive failure, interleaving, retrieval, spacing). Micro-projects on Days 6, 12, 18, 23; defense on Day 24.

Week 1 — Representation & Execution Model

(Days 1–6)

Day 1 — Bits & Two's Complement

Learn: Bases (bin/hex), two's complement, overflow rules.

- Hands-On 1.** Build `packbits`: pack/unpack fields in a 32-bit word via shifts/masks.
2. Implement `saturating_add_i32(a,b)`; test `INT_MAX`, `INT_MIN`, randoms.
3. Journal: why two's complement wins (single adder, sign extension).

Plan-breaker (10–15m): Whiteboard subtraction using two's complement from memory; encode/verify 3 random examples.

Resources:

- Bit Twiddling Hacks — masks/bit tricks: <https://graphics.stanford.edu/~seander/bithacks.html>
 - Two's complement (background): https://en.wikipedia.org/wiki/Two%27s_complement
-

Day 2 — Endianness & Layout

Learn: little vs big endian; struct alignment/padding; `offsetof`.

- Hands-On 1.** Create a mixed `struct`; dump raw bytes with `xxd -g1 -C`; annotate fields.
2. Implement `u32 from_be32(const u8[4])` and `to_be32`.
3. Parse a toy IPv4 header hex blob; print fields.

Plan-breaker (10–20m): Given a wire hexdump (big-endian), reconstruct host-native fields without `ntohl/htonl`.

Resources:

- Endianness overview: <https://en.wikipedia.org/wiki/Endianness>
 - `xxd(1)`: <https://man7.org/linux/man-pages/man1/xxd.1.html>
 - `hexdump(1)`: <https://man7.org/linux/man-pages/man1/hexdump.1.html>
 - IPv4 header (RFC 791 §3.1): <https://www.rfc-editor.org/rfc/rfc791#section-3.1>
-

Day 3 — Floating-Point (just enough)

Learn: IEEE-754 layout, rounding, why `0.1 + 0.2 ≠ 0.3`.

- Hands-On 1.** Manually encode `0.1f`, decode back; verify with a tiny converter.
2. Show catastrophic cancellation and a stable alternative.
3. Journal: one floating-point rule for future reviews.

Plan-breaker (10–15m): Predict equality for `{0.1f*3, 0.3f}` before running; explain discrepancy.

Resources:

- Goldberg — What Every CS Should Know About FP: <https://people.eecs.berkeley.edu/~wkahan/ieee754status/754story.html>
 - Floating-Point Guide: <https://floating-point-gui.de/>
 - Float visualizer: <https://float.exposed/>
-

Day 4 — Gates → ALU → State

Learn: combinational vs sequential logic; registers/clock; adders; fetch–decode–execute.

Hands-On 1. Build 1-bit full adder, then 4-bit ripple (logic sim or code).
2. Emulate a 4-bit adder using **only** bitwise ops (no `+`). Compare carries.

Plan-breaker (10–15m): Step-trace a “paper CPU” (PC, IR, flags) for 5 instructions; update flags manually.

Resources:

- Nand2Tetris (Part I): <https://www.nand2tetris.org/>
 - CircuitVerse simulator: <https://circuitverse.org/>
 - Instruction cycle (overview): https://en.wikipedia.org/wiki/Instruction_cycle
-

Day 5 — ALU Ops & Flags

Learn: add/sub/and/or/xor/shifts; ZF/SF/CF/OF; high-level fetch cycle.

Hands-On 1. Disassemble a “mystery” function; annotate flag effects.
2. Write a program triggering OF/CF/ZF intentionally; print state via inline asm or debugger.

Plan-breaker (10–15m): Given straight-line asm (no running), infer flags after each op; then verify in `gdb`.

Resources:

- Intel® SDM (Vol. 1): <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
 - x86 RFLAGS overview: https://wiki.osdev.org/FLAGS_register
-

Day 6 — Micro-Project 1: Bit-Toolkit

Build: CLI `bitkit` with subcommands: `hex2bin`, `bin2hex`, `twoscomp`, `fields view/set`.
Add an input format you didn’t plan for (e.g., binary with underscores) without >20 LoC growth.

Deliverables: program + tests + 1-page write-up: “Two’s complement, alignment & endianness—why it matters.”

Resources:

- `xxd(1)`: <https://man7.org/linux/man-pages/man1/xxd.1.html>
 - `hexdump(1)`: <https://man7.org/linux/man-pages/man1/hexdump.1.html>
-

Week 2 — ISA & Assembly Fundamentals (Days 7–12)

Day 7 — ISA Surface & Disassembly

Learn: x86-64 registers, addressing modes; AArch64 mapping.

Hands-On 1. Disassemble `int add(int,int)`; label args/return regs, prologue/epilogue, flags touched.
2. On AArch64 (if Apple Silicon), compile same C; list 3 structural diffs (args, frame, condition codes).

Plan-breaker (10–15m): Spot 3 compiler idioms (leaf return, tail-call, `xor reg,reg` zeroing).

Resources:

- x86-64 quick ref: <https://en.wikipedia.org/wiki/X86-64>
 - AAPCS64 (Procedure Call Standard): <https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst>
-

Day 8 — Assembler, Objects, Sections, Tools

Learn: assembler → object → linker; `.text/.data/.bss`; symbols/relocation.

Hands-On 1. Write pure-asm program returning exit code 42; build with NASM; verify `$?`.
2. Inspect with `objdump -d -Mintel`, `readelf -S -s`, `nm`—explain sections/symbols.

Plan-breaker (10–20m): Predict the `objdump` shape (labels/bytes) **before** running; then compare.

Resources:

- Jorgensen (x86-64 Assembly; free text): <https://github.com/0xAX/asm>
 - `objdump(1)`: <https://man7.org/linux/man-pages/man1/objdump.1.html>
 - `readelf(1)`: <https://man7.org/linux/man-pages/man1/readelf.1.html>
 - `nm(1)`: <https://man7.org/linux/man-pages/man1/nm.1.html>
-

Day 9 — System V AMD64 ABI (Stack Frames, Arg Regs)

Learn: SysV AMD64 calling convention—arg/return regs, caller/callee-saved, stack alignment.

Hands-On 1. Implement `int add(int,int)` in asm; call from C; step under `gdb` to verify ABI.
2. Add a second function that clobbers caller-saved regs; prove prologue/epilogue correctness.

Plan-breaker (10–20m): Implement without the stack (only caller-saved regs); document ABI safety.

Resources:

- System V AMD64 psABI: <https://gitlab.com/x86-psABIs/x86-64-ABI>
- Calling conventions overview: https://en.wikipedia.org/wiki/X86_calling_conventions

Day 10 — Control Flow & Branchless Tricks

Learn: conditional jumps vs `cmov`; when branchless helps/hurts.

Hands-On 1. `min(a,b)`—branchy vs branchless; measure at $N=1e7$.
2. Replace a loop conditional with arithmetic/`cmov`; keep the faster variant (record inputs).

Plan-breaker (10–15m): Flip input distribution (1% vs 50% taken); record the break-even where branchless loses.

Resources:

- Agner Fog optimization manuals: <https://www.agner.org/optimize/>
 - Branch prediction intuition: <https://blog.cloudflare.com/branch-prediction/>
-

Day 11 — Data Movement & Addressing

Learn: effective addresses `base+index*scale+disp`; `lea`; alignment implications.

Hands-On 1. Implement scalar `dot_product(float*, float*, int)` in asm; verify numerically.
2. AoS vs SoA layouts; time both; explain (stride, cache lines).

Plan-breaker (10–15m): Use `lea` for arithmetic (no mem access) and show a win or no regression vs `add/shl`.

Resources:

- Intel SDM (addressing modes): <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
 - LEA tricks overview: https://en.wikibooks.org/wiki/X86_Assembly/LEA
-

Day 12 — Micro-Project 2: Tiny ASM Library + C Harness

Build: 3 functions (e.g., `add`, `strlen`, `dot_product`) in asm + C tests. Add a `Makefile` with `-Wall -Wextra -O2 -g -fno-omit-frame-pointer`.

Deliverables: annotated `objdump -d -Mintel` PDF/snapshot, `readelf -S -s` output, `gdb` transcript showing prologue/epilogue.

Resources:

- NASM Manual: <https://www.nasm.us/xdoc/2.16.01/html/nasmdoc0.html>
 - Mixing C and assembly (example): <https://cs.lmu.edu/~ray/notes/gasexamples/>
-

Week 3 — Memory Hierarchy & Performance (Days 13–18)

Day 13 — Caches 101

Learn: cache lines, associativity, locality; conflict vs capacity misses.

Hands-On 1. 2D traversal: row-major vs column-major; time both; explain using line size.
2. Size-sweep beyond L1/L2; annotate behavior changes (plot optional).

Plan-breaker (10–20m): Use power-of-two sizes just over L1/L2 to surface conflict misses; hypothesize causes.

Resources:

- CS:APP course site: <https://csapp.cs.cmu.edu/>
 - Drepper — What Every Programmer Should Know About Memory: <https://www.akkadia.org/drepper/cpumemory.pdf>
-

Day 14 — Branches & Data-Dependent Access

Learn: branch prediction basics; mispredict penalties; data-dependent latency.

Hands-On 1. Branchy vs branchless `min()` with skewed distributions; measure.
2. `perf stat` metrics: cycles, branches, branch-misses; relate to runtime.

Plan-breaker (10–15m): Sort input to improve predictability; quantify gains.

Resources:

- Cloudflare primer: <https://blog.cloudflare.com/branch-prediction/>
 - Linux perf (Brendan Gregg): <https://www.brendangregg.com/perf.html>
-

Day 15 — Alignment & SIMD Teaser (no intrinsics)

Learn: alignment requirements; why misalignment hurts.

Hands-On 1. Allocate aligned buffers with `posix_memalign`; measure R/W throughput aligned vs +1B misaligned.
2. Inspect addresses to confirm alignment.

Plan-breaker (10–15m): Deliberately misalign one benchmark and then restore alignment to recover perf.

Resources:

- `posix_memalign(3)`: https://man7.org/linux/man-pages/man3/posix_memalign.3.html
 - C alignment keywords: <https://en.cppreference.com/w/c/language/Alignas>
-

Day 16 — Memory Model & Atomics (conceptual)

Learn: x86-TSO basics; why reordering exists; fences at a high level.

Hands-On 1. Write a racy shared counter; observe nondeterminism; fix with an atomic.
2. Journal: explain happens-before for your fix.

Plan-breaker (10–15m): Write a failing invariant test first, then fix (productive failure).

Resources:

- x86-TSO overview: <https://www.cl.cam.ac.uk/~pes20/weakmemory/cacm.pdf>
 - Memory Barriers (hardware view): <https://www.kernel.org/doc/Documentation/memory-barriers.txt>
-

Day 17 — Perf Counters & Tooling

Learn: `perf stat | record | report`; event selection; reading results.

Hands-On 1. Re-measure Week-2 functions across sizes; capture cycles/op, branch-miss%, LLC-miss%.
2. Keep **five metrics max** (cycles, branches, branch-misses, LLC-loads, LLC-misses) and decide **one** change.

Plan-breaker (10–15m): Explain one misleading metric and how to validate it with another view.

Resources:

- `perf stat`: <https://man7.org/linux/man-pages/man1/perf-stat.1.html>
 - `perf record/report`: <https://man7.org/linux/man-pages/man1/perf-record.1.html>
-

Day 18 — Micro-Project 3: Cache-Aware Dot Product

Build: Block/chunk inputs to improve locality with **< 10 LoC** changed. Provide a small chart (size vs throughput) and a 5-bullet “why it improved.”

Deliverables: code, `perf stat` before/after, 1-page write-up.

Resources:

- Cache blocking (intro): https://en.wikipedia.org/wiki/Loop_nest_optimization#Blocking
-

Week 4 — OS Interface, Binaries, Integration (Days 19–24)

Day 19 — Raw Syscalls & Process Layout (no libc)

Learn: user↔kernel boundary; syscall ABI; process memory map (text/data/bss/heap/stack).

Hands-On 1. Implement `write` and `exit` via raw syscalls in asm (Linux x86-64).
2. If it fails, debug **only** with `strace -f -tt -y`; save transcript to the journal.

Plan-breaker (10–15m): Print using `writew` scatter-gather; compare calls/bytes vs simple `write` loop.

Resources:

- `write(2)`: <https://man7.org/linux/man-pages/man2/write.2.html>
 - `syscall(2)`: <https://man7.org/linux/man-pages/man2/syscall.2.html>
 - `strace(1)`: <https://man7.org/linux/man-pages/man1/strace.1.html>
-

Day 20 — ELF, PLT/GOT, Static vs Dynamic Link

Learn: sections vs segments; dynamic linking; PLT/GOT; static vs dynamic trade-offs.

Hands-On 1. Build the same program dynamically and statically; compare size and `readelf -d / ldd`.
2. Find a PLT entry in `objdump -d` and trace the first call (lazy binding path).

Plan-breaker (10–20m): Write three concrete trade-offs (size, relocation time, patching/updating).

Resources:

- A look at dynamic linking (LWN): <https://lwn.net/Articles/192624/>
 - ELF gABI: <https://refspecs.linuxfoundation.org/elf/gabi4+/contents.html>
-

Day 21 — Disassembly to Understand (`-O0` vs `-O2` vs `-Os`)

Learn: how compilers lower code; optimization levels.

Hands-On 1. Compile a medium C function at `-O0/-O2/-Os`; `objdump -d -Mintel -S`; annotate differences.
2. Pick a level for the capstone; justify in 5–7 sentences.

Plan-breaker (10–15m): Use Compiler Explorer to compare GCC vs Clang on the same snippet; note one surprise.

Resources:

- Compiler Explorer: <https://godbolt.org/>
 - `objdump(1)`: <https://man7.org/linux/man-pages/man1/objdump.1.html>
-

Day 22 — Interop & API Hardening

Learn: C↔asm boundary; clobber lists; reentrancy; input validation.

Hands-On 1. Wrap asm functions with a C API; validate pointers/lengths; return error codes.
2. Add one negative test per branch; inspect stack/regs under `gdb` on failure.

Plan-breaker (10–15m): Inject an invalid pointer and show early, predictable rejection.

Resources:

- System V AMD64 psABI: <https://gitlab.com/x86-psABIs/x86-64-ABI>
 - GDB manual: <https://sourceware.org/gdb/current/onlinedocs/gdb/>
-

Day 23 — Capstone Build: grep-lite

Build - Chunked file I/O via **raw syscalls** (`open/read/write/close`).

- Inner search loop in asm (naive substring search).
- No libc dependency in the I/O path.

Deliverables: binary + README explaining ISA choices, calling convention, cache considerations, and how you measured.

Stretch: add `-n` (line numbers) and `-i` (case-insensitive).

Resources:

- `open(2)`: <https://man7.org/linux/man-pages/man2/open.2.html>
 - `read(2)`: <https://man7.org/linux/man-pages/man2/read.2.html>
 - `write(2)`: <https://man7.org/linux/man-pages/man2/write.2.html>
 - `close(2)`: <https://man7.org/linux/man-pages/man2/close.2.html>
 - `strace(1)` advanced: <https://man7.org/linux/man-pages/man1/strace.1.html>
-

Day 24 — Capstone Review & Oral Defense (60–90m)

Do - Whiteboard: source → object → ELF → process → CPU → caches.

- Defend the capstone design: where are cycles, where are misses, where could SIMD help?
- List the next five optimizations and what to measure first.

Resources:

- CS:APP index (linking/machine code/memory): <https://csapp.cs.cmu.edu/>
 - Intel SDM: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
-

AArch64 Variant (Apple Silicon / ARM Linux)

- Use AAPCS64 calling convention (x0–x7 args, x29 FP, x30 LR).
 - Syscall numbers & sequences differ; mirror Day-19 with the AArch64 table for your OS.
 - Prefer `lldb` / `llvm-objdump` on macOS.
-

Notes & Guardrails

- **Timebox:** if a Hands-On step exceeds 90m, record the blocker and move on; revisit on Day 24.
- **Measure or it didn't happen:** every optimization must show up in numbers (cycles/op or miss%).
- **Plan-breaker:** recommended 3–4 times/week for desirable difficulty.